

Experiment 9

Module Name: Unit Test Frameworks

Experiment Name: PyTest Integration with HTML Reporting

Problem statement: Convert your setup to run via PyTest.

Objective: Migrate the automation scripts into the pytest framework. Utilize fixtures to handle WebDriver initialization and teardown, and generate a comprehensive HTML execution report.

Tools, software, and concepts used: Python 3, Selenium WebDriver, pytest framework, pytest-html plugin, Fixtures (@pytest.fixture), Assertions.

Source code or implementation steps:

1. Create any Folder inside that create 'tests' folder.
2. Now inside 'tests' folder create 'conftest.py' and write this

```
import pytest
from selenium import webdriver

@pytest.fixture(scope="function")
def setup_driver():
    # Setup: Initialize and configure the browser
    driver = webdriver.Chrome()
    driver.maximize_window()
    driver.get("https://www.saucedemo.com/")

    yield driver # Yield temporarily pauses here and hands the browser to your
test

    # Teardown: Closes the browser after the test finishes
    driver.quit()
```

3. Now inside 'tests' folder create another file 'test_pytest_login.py' and write this

```
from selenium.webdriver.common.by import By

def test_valid_login(setup_driver):
    driver = setup_driver

    driver.find_element(By.ID, "user-name").send_keys("standard_user")
    driver.find_element(By.ID, "password").send_keys("secret_sauce")
    driver.find_element(By.ID, "login-button").click()

    assert "inventory.html" in driver.current_url

def test_invalid_login(setup_driver):
    driver = setup_driver
```

```

driver.find_element(By.ID, "user-name").send_keys("locked_out_user")
driver.find_element(By.ID, "password").send_keys("secret_sauce")
driver.find_element(By.ID, "login-button").click()

error_msg = driver.find_element(By.CSS_SELECTOR, "h3[data-
test='error']").text
assert "locked out" in error_msg

```

4. Now in your created folder go to cmd and write this

```
>>> pytest tests/test_pytest_login.py --html=reports/test_report.html
```

Output with clearly labelled screenshots:

```

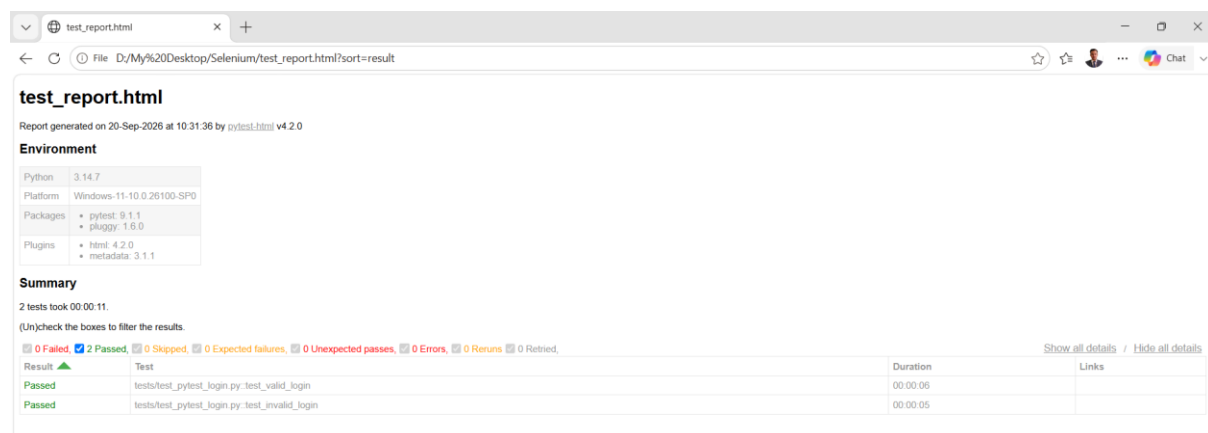
D:\My Desktop\Selenium>pytest tests/test_pytest_login.py --html=reports/test_report.html
===== test session starts =====
platform win32 -- Python 3.14.7, pytest-9.1.1, pluggy-1.6.0
rootdir: D:\My Desktop\Selenium
plugins: html-4.2.0, metadata-3.1.1
collected 2 items

tests\test_pytest_login.py ..                                     [100%]

----- Generated html report: file:///D:/My%20Desktop/Selenium/reports/test_report.html -----
===== 2 passed in 11.49s =====

```

Screenshot 1: Terminal output showing PyTest collecting and executing 2 items



test_report.html

Report generated on 20-Sep-2026 at 10:31:36 by pytest-html v4.2.0

Environment

Python	3.14.7
Platform	Windows-11-10.0.26100-SP0
Packages	- pytest: 9.1.1 - pluggy: 1.6.0
Plugins	- html: 4.2.0 - metadata: 3.1.1

Summary

2 tests took 00:00:11.

(Un)check the boxes to filter the results.

0 Failed, 2 Passed, 0 Skipped, 0 Expected failures, 0 Unexpected passes, 0 Errors, 0 Retuns, 0 Retried.

Show all details / Hide all details

Result	Test	Duration	Links
Passed	tests/test_pytest_login.py::test_valid_login	00:00:06	
Passed	tests/test_pytest_login.py::test_invalid_login	00:00:05	

Screenshot 2: The generated test_report.html file open in a browser showing the execution summary

Brief Result: Pytest successfully identified the two test functions, utilized the yield fixture to manage the browser lifecycle seamlessly around each test, and generated an HTML report.

Observation: Unlike standard Python scripts which execute top-to-bottom, pytest automatically discovers and runs any function prefixed with test_. The conftest.py file allows fixtures to be shared globally across all test files.

Conclusion: Adopting a dedicated unit testing framework like pytest is mandatory for building production-ready automation. It standardizes execution, automates repetitive setup/teardown, and provides the reporting artifacts required by management.